```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>VoxPoll — Democratic Voting Platform</title>
<link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=DM+San
s:ital,opsz,wght@0,9..40,300;0,9..40,400;0,9..40,500;1,9..40,300&display=swap"
rel="stylesheet">
<style>
  :root {
    --bg: #0a0a0f;
    --surface: #12121a;
    --surface2: #1a1a26;
    --surface3: #22223a;
    --border: #2a2a40;
    --accent: #6c63ff;
    --accent2: #ff6584;
    --accent3: #43e97b;
    --accent4: #f7971e;
    --text: #f0f0fa;
    --text2: #9090b0;
    --text3: #5a5a7a;
    --danger: #ff4757;
    --warning: #ffa502;
    --success: #2ed573;
    --radius: 16px;
    --radius-sm: 10px;
    --shadow: 0 8px 32px rgba(0,0,0,0.4);
    --shadow-accent: 0 0 40px rgba(108,99,255,0.15);
  }

  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

  html { scroll-behavior: smooth; }

  body {
    font-family: 'DM Sans', sans-serif;
    background: var(--bg);
    color: var(--text);
    min-height: 100vh;
    overflow-x: hidden;
  }
```

```
/* Background mesh */
body::before {
  content: '';
  position: fixed; inset: 0;
  background:
    radial-gradient(ellipse 60% 40% at 20% 10%, rgba(108,99,255,0.08) 0%, transparent 60%),
    radial-gradient(ellipse 40% 50% at 80% 80%, rgba(255,101,132,0.06) 0%, transparent
60%),
    radial-gradient(ellipse 50% 30% at 60% 30%, rgba(67,233,123,0.04) 0%, transparent 60%);
  pointer-events: none; z-index: 0;
}

/* ===== SCROLLBAR ===== */
::-webkit-scrollbar { width: 6px; }
::-webkit-scrollbar-track { background: var(--bg); }
::-webkit-scrollbar-thumb { background: var(--border); border-radius: 3px; }

/* ===== TYPOGRAPHY ===== */
h1, h2, h3, h4 { font-family: 'Syne', sans-serif; font-weight: 700; }

/* ===== LAYOUT ===== */
#app { position: relative; z-index: 1; min-height: 100vh; }

/* ===== SCREENS ===== */
.screen { display: none; }
.screen.active { display: flex; flex-direction: column; min-height: 100vh; }

/* ===== LOGIN SCREEN ===== */
#login-screen {
  align-items: center; justify-content: center;
  padding: 24px; background: var(--bg);
}

.login-card {
  width: 100%; max-width: 440px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 24px;
  padding: 48px 40px;
  box-shadow: var(--shadow), var(--shadow-accent);
  animation: fadeUp 0.5s ease;
}
```

```
.brand {
  display: flex; align-items: center; gap: 12px;
  margin-bottom: 36px;
}

.brand-icon {
  width: 48px; height: 48px;
  background: linear-gradient(135deg, var(--accent), var(--accent2));
  border-radius: 14px;
  display: flex; align-items: center; justify-content: center;
  font-size: 22px;
}

.brand-name { font-family: 'Syne', sans-serif; font-size: 26px; font-weight: 800; }
.brand-name span { color: var(--accent); }

.login-tabs {
  display: flex; background: var(--surface2); border-radius: var(--radius-sm);
  padding: 4px; margin-bottom: 28px; gap: 4px;
}

.tab-btn {
  flex: 1; padding: 10px; border: none; background: transparent;
  color: var(--text2); font-family: 'DM Sans', sans-serif; font-size: 14px;
  font-weight: 500; border-radius: 8px; cursor: pointer; transition: all 0.2s;
}
.tab-btn.active { background: var(--accent); color: white; }

.form-group { margin-bottom: 18px; }
.form-label { display: block; font-size: 13px; font-weight: 500; color: var(--text2); margin-bottom:
8px; }

.form-input {
  width: 100%; padding: 13px 16px;
  background: var(--surface2); border: 1.5px solid var(--border);
  border-radius: var(--radius-sm); color: var(--text);
  font-family: 'DM Sans', sans-serif; font-size: 15px;
  transition: all 0.2s; outline: none;
}
.form-input:focus { border-color: var(--accent); box-shadow: 0 0 0 3px rgba(108,99,255,0.15); }
.form-input::placeholder { color: var(--text3); }

.role-selector {
  display: flex; gap: 10px; margin-bottom: 24px;
```

```
  }

  .role-option {
    flex: 1; padding: 12px 8px; border: 1.5px solid var(--border);
    background: var(--surface2); border-radius: var(--radius-sm);
    cursor: pointer; text-align: center; transition: all 0.2s;
  }
  .role-option.selected { border-color: var(--accent); background: rgba(108,99,255,0.1); }
  .role-option .role-icon { font-size: 20px; margin-bottom: 4px; }
  .role-option .role-name { font-size: 12px; font-weight: 500; color: var(--text2); }
  .role-option.selected .role-name { color: var(--accent); }

  /* ===== BUTTONS ===== */
  .btn {
    display: inline-flex; align-items: center; justify-content: center; gap: 8px;
    padding: 12px 20px; border: none; border-radius: var(--radius-sm);
    font-family: 'DM Sans', sans-serif; font-size: 15px; font-weight: 500;
    cursor: pointer; transition: all 0.2s; text-decoration: none;
    white-space: nowrap; position: relative; overflow: hidden;
  }
  .btn:disabled { opacity: 0.5; cursor: not-allowed; }

  .btn-primary {
    background: linear-gradient(135deg, var(--accent), #8b5cf6);
    color: white; width: 100%;
    box-shadow: 0 4px 20px rgba(108,99,255,0.3);
  }
  .btn-primary:hover:not(:disabled) { transform: translateY(-1px); box-shadow: 0 6px 28px
rgba(108,99,255,0.4); }
  .btn-primary:active { transform: translateY(0); }

  .btn-secondary {
    background: var(--surface2); color: var(--text); border: 1.5px solid var(--border);
  }
  .btn-secondary:hover { border-color: var(--accent); color: var(--accent); }

  .btn-danger { background: rgba(255,71,87,0.15); color: var(--danger); border: 1.5px solid
rgba(255,71,87,0.3); }
  .btn-danger:hover { background: rgba(255,71,87,0.25); }

  .btn-success { background: rgba(46,213,115,0.15); color: var(--success); border: 1.5px solid
rgba(46,213,115,0.3); }
  .btn-success:hover { background: rgba(46,213,115,0.25); }
```

```
.btn-icon { padding: 10px; width: 40px; height: 40px; border-radius: 10px; }

.btn-sm { padding: 8px 14px; font-size: 13px; }

/* ===== MAIN APP ===== */
#main-screen { flex-direction: row; }

/* ===== SIDEBAR ===== */
.sidebar {
  width: 260px; min-height: 100vh; background: var(--surface);
  border-right: 1px solid var(--border);
  display: flex; flex-direction: column;
  position: fixed; left: 0; top: 0; bottom: 0; z-index: 100;
  transition: transform 0.3s ease;
}

.sidebar-header {
  padding: 28px 24px 20px;
  border-bottom: 1px solid var(--border);
}

.sidebar-brand { display: flex; align-items: center; gap: 10px; }
.sidebar-brand-icon {
  width: 36px; height: 36px;
  background: linear-gradient(135deg, var(--accent), var(--accent2));
  border-radius: 10px; display: flex; align-items: center; justify-content: center; font-size: 16px;
}
.sidebar-brand-name { font-family: 'Syne', sans-serif; font-size: 20px; font-weight: 800; }
.sidebar-brand-name span { color: var(--accent); }

.sidebar-user {
  margin: 16px 0 0; padding: 12px;
  background: var(--surface2); border-radius: var(--radius-sm);
  display: flex; align-items: center; gap: 10px;
}
.user-avatar {
  width: 36px; height: 36px; border-radius: 10px;
  display: flex; align-items: center; justify-content: center;
  font-weight: 700; font-size: 14px; color: white; flex-shrink: 0;
}
.user-info { min-width: 0; }
.user-name { font-size: 14px; font-weight: 600; white-space: nowrap; overflow: hidden;
text-overflow: ellipsis; }
.user-role { font-size: 11px; color: var(--text2); margin-top: 1px; }
```

```
.sidebar-nav { flex: 1; padding: 16px 12px; overflow-y: auto; }

.nav-section-label {
  font-size: 10px; font-weight: 600; letter-spacing: 1.5px;
  color: var(--text3); text-transform: uppercase; padding: 8px 12px 4px; margin-top: 8px;
}

.nav-item {
  display: flex; align-items: center; gap: 12px;
  padding: 11px 12px; border-radius: var(--radius-sm);
  color: var(--text2); cursor: pointer; transition: all 0.2s; margin-bottom: 2px;
  font-size: 14px; font-weight: 500;
}
.nav-item:hover { background: var(--surface2); color: var(--text); }
.nav-item.active { background: rgba(108,99,255,0.15); color: var(--accent); }
.nav-item .nav-icon { font-size: 18px; width: 22px; text-align: center; flex-shrink: 0; }
.nav-badge {
  margin-left: auto; background: var(--accent); color: white;
  font-size: 11px; padding: 2px 7px; border-radius: 20px; font-weight: 600;
}

.sidebar-footer {
  padding: 16px 12px;
  border-top: 1px solid var(--border);
}

.sidebar-close { display: none; }

/* ===== MAIN CONTENT ===== */
.main-content {
  margin-left: 260px;
  flex: 1; padding: 32px;
  max-width: calc(100% - 260px);
}

.page { display: none; }
.page.active { display: block; animation: fadeUp 0.3s ease; }

/* ===== TOP BAR ===== */
.topbar {
  display: flex; align-items: center; justify-content: space-between;
  margin-bottom: 32px; gap: 16px; flex-wrap: wrap;
}
```

```
.topbar-left {}
.page-title { font-size: 28px; font-weight: 800; font-family: 'Syne', sans-serif; }
.page-subtitle { color: var(--text2); font-size: 14px; margin-top: 4px; }

.topbar-right { display: flex; gap: 10px; align-items: center; }

.hamburger {
  display: none; background: var(--surface2); border: 1.5px solid var(--border);
  border-radius: 10px; padding: 8px 10px; cursor: pointer; font-size: 18px;
  color: var(--text);
}

/* ===== STATS GRID ===== */
.stats-grid {
  display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px; margin-bottom: 28px;
}

.stat-card {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: var(--radius); padding: 20px;
  transition: all 0.2s;
}
.stat-card:hover { border-color: var(--accent); transform: translateY(-2px); }
.stat-icon { font-size: 24px; margin-bottom: 12px; }
.stat-value { font-size: 32px; font-weight: 800; font-family: 'Syne', sans-serif; line-height: 1; }
.stat-label { font-size: 13px; color: var(--text2); margin-top: 4px; }

/* ===== POLLS GRID ===== */
.polls-grid {
  display: grid; grid-template-columns: repeat(auto-fill, minmax(340px, 1fr));
  gap: 20px;
}

.poll-card {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: var(--radius); padding: 24px;
  transition: all 0.25s; position: relative; overflow: hidden;
}
.poll-card::before {
  content: '';
  position: absolute; top: 0; left: 0; right: 0; height: 3px;
  background: linear-gradient(90deg, var(--accent), var(--accent2));
```

```
  }
  .poll-card:hover { border-color: var(--accent); box-shadow: var(--shadow-accent); transform:
translateY(-2px); }

  .poll-card-header {
    display: flex; justify-content: space-between; align-items: flex-start;
    margin-bottom: 12px; gap: 12px;
  }

  .poll-title { font-size: 17px; font-weight: 700; font-family: 'Syne', sans-serif; line-height: 1.3; }
  .poll-desc { font-size: 13px; color: var(--text2); margin-top: 6px; line-height: 1.5; }

  .poll-actions { display: flex; gap: 6px; flex-shrink: 0; }

  .status-badge {
    display: inline-flex; align-items: center; gap: 6px;
    padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600;
    letter-spacing: 0.5px; text-transform: uppercase;
  }
  .status-badge.active { background: rgba(46,213,115,0.15); color: var(--success); }
  .status-badge.active::before { content: ''; width: 6px; height: 6px; border-radius: 50%;
background: var(--success); animation: pulse 1.5s infinite; }
  .status-badge.closed { background: rgba(90,90,122,0.2); color: var(--text3); }
  .status-badge.scheduled { background: rgba(247,151,30,0.15); color: var(--warning); }
  .status-badge.draft { background: rgba(108,99,255,0.15); color: var(--accent); }

  .poll-meta {
    display: flex; gap: 16px; margin: 14px 0;
    font-size: 12px; color: var(--text2);
    flex-wrap: wrap;
  }
  .poll-meta-item { display: flex; align-items: center; gap: 5px; }

  .poll-progress {
    background: var(--surface2); border-radius: 4px; height: 4px;
    margin: 16px 0; overflow: hidden;
  }
  .poll-progress-bar {
    height: 100%; border-radius: 4px;
    background: linear-gradient(90deg, var(--accent), var(--accent2));
    transition: width 0.5s ease;
  }

  .poll-footer {
```

```
  display: flex; justify-content: space-between; align-items: center;
  margin-top: 16px; flex-wrap: wrap; gap: 10px;
}

.vote-count { font-size: 13px; color: var(--text2); }
.vote-count strong { color: var(--text); }

/* ===== MODAL ===== */
.modal-overlay {
  display: none; position: fixed; inset: 0;
  background: rgba(0,0,0,0.7); backdrop-filter: blur(4px);
  z-index: 1000; align-items: center; justify-content: center;
  padding: 20px;
}
.modal-overlay.open { display: flex; }

.modal {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: 24px; width: 100%; max-width: 580px;
  max-height: 90vh; overflow-y: auto;
  box-shadow: var(--shadow), var(--shadow-accent);
  animation: modalIn 0.3s ease;
}

.modal-header {
  padding: 28px 28px 0;
  display: flex; justify-content: space-between; align-items: center;
  position: sticky; top: 0; background: var(--surface);
  z-index: 1; padding-bottom: 20px; border-bottom: 1px solid var(--border);
}
.modal-title { font-size: 22px; font-weight: 800; font-family: 'Syne', sans-serif; }

.modal-close {
  background: var(--surface2); border: 1.5px solid var(--border);
  width: 36px; height: 36px; border-radius: 10px;
  display: flex; align-items: center; justify-content: center;
  cursor: pointer; font-size: 18px; color: var(--text2); transition: all 0.2s;
}
.modal-close:hover { background: var(--danger); border-color: var(--danger); color: white; }

.modal-body { padding: 24px 28px; }
.modal-footer { padding: 0 28px 28px; display: flex; gap: 12px; justify-content: flex-end; }

/* ===== OPTIONS BUILDER ===== */
```

```
.options-list { display: flex; flex-direction: column; gap: 10px; margin: 12px 0; }

.option-item {
  display: flex; gap: 10px; align-items: center;
}
.option-item .form-input { flex: 1; }

.add-option-btn {
  display: flex; align-items: center; gap: 8px;
  padding: 10px 14px; border: 1.5px dashed var(--border);
  background: transparent; border-radius: var(--radius-sm);
  color: var(--text2); cursor: pointer; font-family: 'DM Sans', sans-serif;
  font-size: 14px; transition: all 0.2s; width: 100%;
}
.add-option-btn:hover { border-color: var(--accent); color: var(--accent); }

/* ===== VOTE SCREEN ===== */
.vote-options { display: flex; flex-direction: column; gap: 12px; margin: 20px 0; }

.vote-option {
  padding: 16px 20px;
  border: 1.5px solid var(--border);
  border-radius: var(--radius-sm);
  cursor: pointer; transition: all 0.2s;
  display: flex; align-items: center; gap: 14px;
  position: relative; overflow: hidden;
}
.vote-option:hover { border-color: var(--accent); background: rgba(108,99,255,0.05); }
.vote-option.selected { border-color: var(--accent); background: rgba(108,99,255,0.12); }

.vote-radio {
  width: 20px; height: 20px; border-radius: 50%;
  border: 2px solid var(--border); flex-shrink: 0;
  transition: all 0.2s; display: flex; align-items: center; justify-content: center;
}
.vote-option.selected .vote-radio { border-color: var(--accent); background: var(--accent); }
.vote-option.selected .vote-radio::after { content: ''; width: 8px; height: 8px; border-radius: 50%;
background: white; }

.vote-option-text { font-size: 15px; font-weight: 500; }

/* ===== RESULTS ===== */
.results-container { margin-top: 12px; }
```

```
.result-row {
  margin-bottom: 16px;
}

.result-header {
  display: flex; justify-content: space-between; align-items: center;
  margin-bottom: 6px;
}

.result-label { font-size: 14px; font-weight: 500; }
.result-pct { font-size: 14px; font-weight: 700; color: var(--accent); }
.result-count { font-size: 12px; color: var(--text2); }

.result-bar-bg {
  height: 10px; background: var(--surface2); border-radius: 5px; overflow: hidden;
}

.result-bar {
  height: 100%; border-radius: 5px; transition: width 1s cubic-bezier(0.16, 1, 0.3, 1);
  position: relative;
}

/* ===== CHARTS ===== */
.chart-container { display: flex; gap: 24px; flex-wrap: wrap; align-items: flex-start; margin-top:
20px; }
.chart-wrap { flex: 1; min-width: 200px; }
.chart-legend { display: flex; flex-direction: column; gap: 8px; flex: 1; min-width: 160px; }
.legend-item { display: flex; align-items: center; gap: 10px; font-size: 13px; }
.legend-dot { width: 12px; height: 12px; border-radius: 50%; flex-shrink: 0; }

/* PIE CHART */
.pie-chart { width: 160px; height: 160px; border-radius: 50%; position: relative; }

/* ===== ANALYTICS ===== */
.analytics-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)); gap:
20px; }

.analytics-card {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: var(--radius); padding: 24px;
}
.analytics-card-title { font-size: 16px; font-weight: 700; margin-bottom: 16px; font-family: 'Syne',
sans-serif; }
```

```
/* ===== COMPARISON ===== */
.comparison-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-top: 20px; }

/* ===== TOAST ===== */
.toast-container {
  position: fixed; top: 24px; right: 24px; z-index: 9999;
  display: flex; flex-direction: column; gap: 10px; pointer-events: none;
}

.toast {
  padding: 14px 18px; border-radius: var(--radius-sm);
  font-size: 14px; font-weight: 500;
  display: flex; align-items: center; gap: 10px;
  animation: slideInRight 0.3s ease, fadeOut 0.3s ease 2.7s forwards;
  pointer-events: auto; max-width: 340px; box-shadow: var(--shadow);
  border: 1px solid transparent;
}
.toast.success { background: rgba(46,213,115,0.15); color: var(--success); border-color:
rgba(46,213,115,0.3); }
.toast.error { background: rgba(255,71,87,0.15); color: var(--danger); border-color:
rgba(255,71,87,0.3); }
.toast.info { background: rgba(108,99,255,0.15); color: var(--accent); border-color:
rgba(108,99,255,0.3); }
.toast.warning { background: rgba(255,165,2,0.15); color: var(--warning); border-color:
rgba(255,165,2,0.3); }

/* ===== TIMER DISPLAY ===== */
.countdown {
  display: inline-flex; gap: 8px; align-items: center;
  font-size: 12px; font-family: 'Syne', sans-serif;
  color: var(--warning); font-weight: 600;
}
.countdown.expired { color: var(--danger); }

/* ===== EMPTY STATE ===== */
.empty-state {
  display: flex; flex-direction: column; align-items: center; justify-content: center;
  padding: 60px 20px; text-align: center;
  color: var(--text2);
}
.empty-icon { font-size: 52px; margin-bottom: 16px; opacity: 0.5; }
.empty-title { font-size: 20px; font-weight: 700; font-family: 'Syne', sans-serif; color: var(--text);
margin-bottom: 8px; }
.empty-desc { font-size: 14px; line-height: 1.6; max-width: 300px; }
```

```
/* ===== DIVIDER ===== */
.divider { border: none; border-top: 1px solid var(--border); margin: 20px 0; }

/* ===== TABS ===== */
.view-tabs {
  display: flex; gap: 8px; margin-bottom: 24px;
  background: var(--surface); border: 1px solid var(--border);
  border-radius: var(--radius-sm); padding: 4px;
  width: fit-content;
}
.view-tab {
  padding: 8px 16px; border: none; border-radius: 8px;
  background: transparent; color: var(--text2);
  font-family: 'DM Sans', sans-serif; font-size: 14px; font-weight: 500;
  cursor: pointer; transition: all 0.2s;
}
.view-tab.active { background: var(--accent); color: white; }

/* ===== SEARCH ===== */
.search-box {
  position: relative; flex: 1; max-width: 280px;
}
.search-icon {
  position: absolute; left: 14px; top: 50%; transform: translateY(-50%);
  color: var(--text3); font-size: 16px; pointer-events: none;
}
.search-input {
  width: 100%; padding: 10px 14px 10px 40px;
  background: var(--surface2); border: 1.5px solid var(--border);
  border-radius: var(--radius-sm); color: var(--text);
  font-family: 'DM Sans', sans-serif; font-size: 14px;
  transition: all 0.2s; outline: none;
}
.search-input:focus { border-color: var(--accent); }
.search-input::placeholder { color: var(--text3); }

/* ===== FILTER BAR ===== */
.filter-bar {
  display: flex; gap: 12px; margin-bottom: 20px; flex-wrap: wrap; align-items: center;
}

.filter-select {
  padding: 9px 14px; background: var(--surface2); border: 1.5px solid var(--border);
```

```
  border-radius: var(--radius-sm); color: var(--text);
  font-family: 'DM Sans', sans-serif; font-size: 14px; cursor: pointer; outline: none;
  transition: all 0.2s;
}
.filter-select:focus { border-color: var(--accent); }

/* ===== INSTRUCTIONS BOX ===== */
.info-box {
  background: rgba(108,99,255,0.08); border: 1px solid rgba(108,99,255,0.2);
  border-radius: var(--radius-sm); padding: 14px 16px;
  font-size: 13px; line-height: 1.6; color: var(--text2);
  margin-bottom: 20px;
}
.info-box strong { color: var(--accent); }

/* ===== PROGRESS STEP ===== */
.steps-indicator {
  display: flex; gap: 0; margin-bottom: 28px; align-items: center;
}
.step {
  display: flex; align-items: center; gap: 8px; flex: 1;
}
.step-circle {
  width: 32px; height: 32px; border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  font-size: 13px; font-weight: 700; flex-shrink: 0;
  border: 2px solid var(--border); color: var(--text2);
  transition: all 0.3s;
}
.step.done .step-circle { background: var(--success); border-color: var(--success); color: white;
}
.step.active .step-circle { background: var(--accent); border-color: var(--accent); color: white; }
.step-label { font-size: 12px; color: var(--text2); display: none; }
.step-line { flex: 1; height: 2px; background: var(--border); margin: 0 8px; }
.step.done + .step-line,
.step.active + .step-line { background: var(--accent); }

/* ===== SWITCH ===== */
.switch-wrap { display: flex; align-items: center; gap: 12px; margin-bottom: 16px; }
.switch {
  width: 44px; height: 24px; background: var(--border);
  border-radius: 12px; position: relative; cursor: pointer;
  transition: background 0.2s; flex-shrink: 0;
}
```

```
.switch.on { background: var(--accent); }
.switch::after {
  content: ''; position: absolute; top: 3px; left: 3px;
  width: 18px; height: 18px; border-radius: 50%; background: white;
  transition: transform 0.2s;
}
.switch.on::after { transform: translateX(20px); }
.switch-label { font-size: 14px; color: var(--text2); }

/* ===== SKELETON LOADER ===== */
.skeleton {
  background: linear-gradient(90deg, var(--surface2) 25%, var(--surface3) 50%, var(--surface2)
75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
  border-radius: 8px;
}

/* ===== OVERLAY SIDEBAR ===== */
.sidebar-overlay {
  display: none; position: fixed; inset: 0;
  background: rgba(0,0,0,0.5); z-index: 99;
}
.sidebar-overlay.open { display: block; }

/* ===== ANIMATIONS ===== */
@keyframes fadeUp {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
@keyframes modalIn {
  from { opacity: 0; transform: scale(0.95) translateY(10px); }
  to { opacity: 1; transform: scale(1) translateY(0); }
}
@keyframes slideInRight {
  from { opacity: 0; transform: translateX(40px); }
  to { opacity: 1; transform: translateX(0); }
}
@keyframes fadeOut {
  from { opacity: 1; }
  to { opacity: 0; pointer-events: none; }
}
@keyframes pulse {
  0%, 100% { opacity: 1; transform: scale(1); }
```

```
  50% { opacity: 0.6; transform: scale(1.2); }
}
@keyframes shimmer {
  from { background-position: 200% 0; }
  to { background-position: -200% 0; }
}

/* ===== RESPONSIVE ===== */
@media (max-width: 900px) {
  .sidebar {
    transform: translateX(-100%);
  }
  .sidebar.open {
    transform: translateX(0);
  }
  .sidebar-close { display: flex; }
  .main-content { margin-left: 0; max-width: 100%; padding: 20px 16px; }
  .hamburger { display: flex; }
  .polls-grid { grid-template-columns: 1fr; }
  .comparison-grid { grid-template-columns: 1fr; }
  .stats-grid { grid-template-columns: repeat(2, 1fr); }
  .topbar { flex-wrap: wrap; }
  .filter-bar { flex-wrap: wrap; }
}

@media (max-width: 480px) {
  .login-card { padding: 32px 24px; }
  .stats-grid { grid-template-columns: 1fr 1fr; }
  .modal-body, .modal-header { padding: 20px; }
  .modal-footer { padding: 0 20px 20px; }
  .page-title { font-size: 22px; }
  .comparison-grid { grid-template-columns: 1fr; }
  .analytics-grid { grid-template-columns: 1fr; }
  .view-tabs { flex-wrap: wrap; width: 100%; }
  .modal { border-radius: 20px 20px 0 0; max-height: 95vh; align-self: flex-end; }
  .modal-overlay { align-items: flex-end; padding: 0; }
  .topbar-right .btn:not(.hamburger) .btn-text { display: none; }
}

/* Ensure color palette is available */
.color-0 { background: linear-gradient(135deg, #6c63ff, #8b5cf6); }
.color-1 { background: linear-gradient(135deg, #ff6584, #ff4757); }
.color-2 { background: linear-gradient(135deg, #43e97b, #38f9d7); }
.color-3 { background: linear-gradient(135deg, #f7971e, #ffd200); }
```

```
  .color-4 { background: linear-gradient(135deg, #4facfe, #00f2fe); }
  .color-5 { background: linear-gradient(135deg, #fa709a, #fee140); }
  .color-6 { background: linear-gradient(135deg, #30cfd0, #330867); }
  .color-7 { background: linear-gradient(135deg, #a18cd1, #fbc2eb); }

  .bar-color-0 { background: linear-gradient(90deg, #6c63ff, #8b5cf6); }
  .bar-color-1 { background: linear-gradient(90deg, #ff6584, #ff4757); }
  .bar-color-2 { background: linear-gradient(90deg, #43e97b, #38f9d7); }
  .bar-color-3 { background: linear-gradient(90deg, #f7971e, #ffd200); }
  .bar-color-4 { background: linear-gradient(90deg, #4facfe, #00f2fe); }
  .bar-color-5 { background: linear-gradient(90deg, #fa709a, #fee140); }
  .bar-color-6 { background: linear-gradient(90deg, #30cfd0, #330867); }
  .bar-color-7 { background: linear-gradient(90deg, #a18cd1, #fbc2eb); }

  .dot-color-0 { background: #6c63ff; }
  .dot-color-1 { background: #ff6584; }
  .dot-color-2 { background: #43e97b; }
  .dot-color-3 { background: #f7971e; }
  .dot-color-4 { background: #4facfe; }
  .dot-color-5 { background: #fa709a; }
  .dot-color-6 { background: #30cfd0; }
  .dot-color-7 { background: #a18cd1; }

  /* Confirm dialog */
  .confirm-btns { display: flex; gap: 12px; margin-top: 16px; }
  .confirm-btns .btn { flex: 1; }

  /* Select2-like */
  .multi-select-wrap { position: relative; }

  /* Timeline pill */
  .time-pill {
    display: inline-flex; align-items: center; gap: 6px;
    padding: 5px 10px; background: var(--surface2);
    border: 1px solid var(--border); border-radius: 20px;
    font-size: 12px; color: var(--text2);
  }
  .time-pill.danger { border-color: rgba(255,71,87,0.3); color: var(--danger); background:
rgba(255,71,87,0.08); }
  .time-pill.success { border-color: rgba(46,213,115,0.3); color: var(--success); background:
rgba(46,213,115,0.08); }

</style>
</head>
```

```
<body>
<div id="app">

  <!-- ===== LOGIN SCREEN ===== -->
  <div id="login-screen" class="screen active">
    <div class="login-card">
      <div class="brand">
        <div class="brand-icon">🗳️</div>
        <div class="brand-name">Vox<span>Poll</span></div>
      </div>

      <div class="login-tabs">
        <button class="tab-btn active" onclick="switchLoginTab('login')">Sign In</button>
        <button class="tab-btn" onclick="switchLoginTab('register')">Register</button>
      </div>

      <!-- Login form -->
      <div id="login-form">
        <div class="info-box">
          <strong>Demo Accounts</strong><br>
          Admin: <strong>admin / admin123</strong> — Voter: <strong>voter1 / pass123</strong>
        </div>
        <div class="form-group">
          <label class="form-label">Username</label>
          <input class="form-input" id="login-username" type="text" placeholder="Enter your
username" autocomplete="username">
        </div>
        <div class="form-group" style="margin-bottom:24px">
          <label class="form-label">Password</label>
          <input class="form-input" id="login-password" type="password" placeholder="Enter your
password" autocomplete="current-password">
        </div>
        <button class="btn btn-primary" onclick="doLogin()">
          <span>🔑</span> Sign In
        </button>
      </div>

      <!-- Register form -->
      <div id="register-form" style="display:none">
        <div class="form-group">
          <label class="form-label">Full Name</label>
          <input class="form-input" id="reg-name" type="text" placeholder="John Doe">
        </div>
        <div class="form-group">
```

```
        <label class="form-label">Username</label>
        <input class="form-input" id="reg-username" type="text" placeholder="Choose a
username">
      </div>
      <div class="form-group">
        <label class="form-label">Password</label>
        <input class="form-input" id="reg-password" type="password" placeholder="Create a
password">
      </div>
      <div class="form-label" style="margin-bottom:8px">Account Type</div>
      <div class="role-selector">
        <div class="role-option selected" data-role="voter" onclick="selectRole(this,'voter')">
          <div class="role-icon">🗳️</div>
          <div class="role-name">Voter</div>
        </div>
        <div class="role-option" data-role="admin" onclick="selectRole(this,'admin')">
          <div class="role-icon">⚙️</div>
          <div class="role-name">Admin</div>
        </div>
      </div>
      <button class="btn btn-primary" style="margin-top:8px" onclick="doRegister()">
        <span>✨</span> Create Account
      </button>
    </div>
  </div>
</div>

<!-- ===== MAIN SCREEN ===== -->
<div id="main-screen" class="screen">
  <div class="sidebar-overlay" id="sidebar-overlay" onclick="closeSidebar()"></div>

  <!-- Sidebar -->
  <aside class="sidebar" id="sidebar">
    <div class="sidebar-header">
      <div class="sidebar-brand">
        <div class="sidebar-brand-icon">🗳️</div>
        <div class="sidebar-brand-name">Vox<span>Poll</span></div>
      </div>
      <div class="sidebar-user" id="sidebar-user">
        <div class="user-avatar" id="user-avatar"></div>
        <div class="user-info">
          <div class="user-name" id="user-display-name">User</div>
          <div class="user-role" id="user-display-role">Voter</div>
        </div>
```

```
      </div>
    </div>

    <nav class="sidebar-nav">
      <div class="nav-section-label">Menu</div>
      <div class="nav-item active" onclick="showPage('dashboard')" data-page="dashboard">
        <span class="nav-icon">📊</span> Dashboard
      </div>
      <div class="nav-item" onclick="showPage('polls')" data-page="polls">
        <span class="nav-icon">🗳️</span> All Polls
        <span class="nav-badge" id="active-poll-count">0</span>
      </div>
      <div class="nav-item" onclick="showPage('my-votes')" data-page="my-votes">
        <span class="nav-icon">✅</span> My Votes
      </div>

      <div class="nav-section-label admin-only">Admin</div>
      <div class="nav-item admin-only" onclick="showPage('manage')" data-page="manage">
        <span class="nav-icon">⚙️</span> Manage Polls
      </div>
      <div class="nav-item admin-only" onclick="showPage('analytics')" data-page="analytics">
        <span class="nav-icon">📈</span> Analytics
      </div>
      <div class="nav-item admin-only" onclick="showPage('comparison')"
data-page="comparison">
        <span class="nav-icon">⚖️</span> Compare Polls
      </div>
    </nav>

    <div class="sidebar-footer">
      <button class="btn btn-secondary btn-sm sidebar-close" onclick="closeSidebar()"
style="margin-bottom:8px;width:100%">✕ Close Menu</button>
      <button class="btn btn-secondary btn-sm" style="width:100%" onclick="doLogout()">🚪
Sign Out</button>
    </div>
  </aside>

  <!-- Main Content -->
  <main class="main-content">

    <!-- ===== DASHBOARD ===== -->
    <div id="page-dashboard" class="page active">
      <div class="topbar">
        <div class="topbar-left">
```

```
          <button class="hamburger" onclick="openSidebar()">☰</button>
        </div>
        <div class="topbar-right">
          <div class="search-box">
            <span class="search-icon">🔍</span>
            <input class="search-input" placeholder="Search polls…"
oninput="filterPolls(this.value,'dashboard')">
          </div>
        </div>
      </div>
      <div class="page-title">Dashboard</div>
      <div class="page-subtitle" id="dash-subtitle">Welcome back!</div>

      <div class="stats-grid" style="margin-top:20px">
        <div class="stat-card">
          <div class="stat-icon">🗳️</div>
          <div class="stat-value" id="stat-total">0</div>
          <div class="stat-label">Total Polls</div>
        </div>
        <div class="stat-card">
          <div class="stat-icon" style="color:var(--success)">🟢</div>
          <div class="stat-value" id="stat-active">0</div>
          <div class="stat-label">Active Now</div>
        </div>
        <div class="stat-card">
          <div class="stat-icon" style="color:var(--accent2)">✅</div>
          <div class="stat-value" id="stat-voted">0</div>
          <div class="stat-label">My Votes Cast</div>
        </div>
        <div class="stat-card">
          <div class="stat-icon" style="color:var(--accent4)">📊</div>
          <div class="stat-value" id="stat-votes-total">0</div>
          <div class="stat-label">Total Votes</div>
        </div>
      </div>

      <h2 style="font-size:18px;margin-bottom:16px;font-family:'Syne',sans-serif">Active
Polls</h2>
      <div class="polls-grid" id="dashboard-polls"></div>
    </div>

    <!-- ===== POLLS PAGE ===== -->
    <div id="page-polls" class="page">
      <div class="topbar">
```

```
        <div class="topbar-left">
          <button class="hamburger" onclick="openSidebar()">☰</button>
        </div>
      </div>
      <div class="page-title">All Polls</div>
      <div class="page-subtitle">Browse and participate in polls</div>

      <div class="filter-bar" style="margin-top:20px">
        <div class="search-box" style="max-width:100%;flex:1">
          <span class="search-icon">🔍</span>
          <input class="search-input" placeholder="Search polls…"
oninput="filterPolls(this.value,'polls')">
        </div>
        <select class="filter-select" id="status-filter" onchange="renderPollsPage()">
          <option value="all">All Status</option>
          <option value="active">Active</option>
          <option value="closed">Closed</option>
          <option value="scheduled">Scheduled</option>
        </select>
        <select class="filter-select" id="sort-filter" onchange="renderPollsPage()">
          <option value="newest">Newest First</option>
          <option value="oldest">Oldest First</option>
          <option value="most-votes">Most Votes</option>
        </select>
      </div>

      <div class="polls-grid" id="polls-list"></div>
    </div>

    <!-- ===== MY VOTES ===== -->
    <div id="page-my-votes" class="page">
      <div class="topbar">
        <div class="topbar-left">
          <button class="hamburger" onclick="openSidebar()">☰</button>
        </div>
      </div>
      <div class="page-title">My Votes</div>
      <div class="page-subtitle">Your voting history</div>
      <div class="polls-grid" id="my-votes-list" style="margin-top:20px"></div>
    </div>

    <!-- ===== MANAGE (Admin) ===== -->
    <div id="page-manage" class="page">
      <div class="topbar">
```

```
    <div class="topbar-left">
      <button class="hamburger" onclick="openSidebar()">☰</button>
    </div>
    <div class="topbar-right">
      <button class="btn btn-primary btn-sm" onclick="openCreatePoll()">
        <span>＋</span> <span class="btn-text">New Poll</span>
      </button>
    </div>
  </div>
  <div class="page-title">Manage Polls</div>
  <div class="page-subtitle">Create, edit, and manage your polls</div>
  <div class="polls-grid" id="manage-list" style="margin-top:20px"></div>
</div>

<!-- ===== ANALYTICS (Admin) ===== -->
<div id="page-analytics" class="page">
  <div class="topbar">
    <div class="topbar-left">
      <button class="hamburger" onclick="openSidebar()">☰</button>
    </div>
  </div>
  <div class="page-title">Analytics</div>
  <div class="page-subtitle">Detailed poll insights</div>
  <div id="analytics-content" style="margin-top:20px"></div>
</div>

<!-- ===== COMPARISON (Admin) ===== -->
<div id="page-comparison" class="page">
  <div class="topbar">
    <div class="topbar-left">
      <button class="hamburger" onclick="openSidebar()">☰</button>
    </div>
  </div>
  <div class="page-title">Compare Polls</div>
  <div class="page-subtitle">Side-by-side poll analysis</div>
  <div style="display:flex;gap:12px;margin-top:20px;flex-wrap:wrap;align-items:center">
    <select class="filter-select" id="compare-poll-1" style="flex:1">
      <option value="">Select Poll 1…</option>
    </select>
    <span style="color:var(--text2);font-size:20px">⚖️</span>
    <select class="filter-select" id="compare-poll-2" style="flex:1">
      <option value="">Select Poll 2…</option>
    </select>
    <button class="btn btn-primary btn-sm" onclick="renderComparison()">Compare</button>
```

```
      </div>
      <div id="comparison-content" style="margin-top:24px"></div>
    </div>

  </main>
</div><!-- end main-screen -->

<!-- ===== TOAST CONTAINER ===== -->
<div class="toast-container" id="toast-container"></div>

<!-- ===== POLL MODAL (create/edit) ===== -->
<div class="modal-overlay" id="poll-modal">
  <div class="modal">
    <div class="modal-header">
      <div class="modal-title" id="poll-modal-title">Create Poll</div>
      <div class="modal-close" onclick="closeModal('poll-modal')">✕</div>
    </div>
    <div class="modal-body">
      <div class="steps-indicator">
        <div class="step active" id="step-1"><div class="step-circle">1</div></div>
        <div class="step-line"></div>
        <div class="step" id="step-2"><div class="step-circle">2</div></div>
        <div class="step-line"></div>
        <div class="step" id="step-3"><div class="step-circle">3</div></div>
      </div>

      <!-- Step 1: Basic Info -->
      <div id="modal-step-1">
        <div class="form-group">
          <label class="form-label">Poll Title *</label>
          <input class="form-input" id="poll-title" placeholder="What would you like to vote on?"
maxlength="120">
        </div>
        <div class="form-group">
          <label class="form-label">Description</label>
          <textarea class="form-input" id="poll-desc" placeholder="Provide additional context…"
rows="3" style="resize:vertical"></textarea>
        </div>
        <div class="form-group">
          <label class="form-label">Poll Type</label>
          <select class="form-input" id="poll-type">
            <option value="single">Single Choice</option>
            <option value="multiple">Multiple Choice</option>
          </select>
```

```
  </div>
</div>

<!-- Step 2: Options -->
<div id="modal-step-2" style="display:none">
  <div class="info-box">
    <strong>Add your options</strong><br>
    Add at least 2 options for voters to choose from. You can reorder by dragging.
  </div>
  <div class="options-list" id="options-list"></div>
  <button class="add-option-btn" onclick="addOption()">
    <span>＋</span> Add Option
  </button>
</div>

<!-- Step 3: Scheduling -->
<div id="modal-step-3" style="display:none">
  <div class="switch-wrap">
    <div class="switch" id="schedule-switch" onclick="toggleSchedule()"></div>
    <div class="switch-label">Set start & end time</div>
  </div>
  <div id="schedule-fields" style="display:none">
    <div class="form-group">
      <label class="form-label">Start Time</label>
      <input class="form-input" id="poll-start" type="datetime-local">
    </div>
    <div class="form-group">
      <label class="form-label">End Time</label>
      <input class="form-input" id="poll-end" type="datetime-local">
    </div>
  </div>
  <div class="switch-wrap" style="margin-top:8px">
    <div class="switch on" id="results-switch" onclick="toggleResults()"></div>
    <div class="switch-label">Show results to voters</div>
  </div>
  <div class="switch-wrap">
    <div class="switch on" id="public-switch" onclick="togglePublic()"></div>
    <div class="switch-label">Public poll (visible to all)</div>
  </div>
  <div class="form-group" style="margin-top:8px">
    <label class="form-label">Status</label>
    <select class="form-input" id="poll-status">
      <option value="active">Active (open for voting)</option>
      <option value="draft">Draft (not visible)</option>
```

```
            <option value="closed">Closed</option>
          </select>
        </div>
      </div>
    </div>
    <div class="modal-footer">
      <button class="btn btn-secondary" id="modal-back-btn" onclick="pollModalBack()"
style="display:none">← Back</button>
      <button class="btn btn-primary" id="modal-next-btn" onclick="pollModalNext()"
style="flex:1;max-width:180px">Next →</button>
    </div>
  </div>
</div>

<!-- ===== VOTE MODAL ===== -->
<div class="modal-overlay" id="vote-modal">
  <div class="modal">
    <div class="modal-header">
      <div class="modal-title" id="vote-modal-title">Cast Your Vote</div>
      <div class="modal-close" onclick="closeModal('vote-modal')">✕</div>
    </div>
    <div class="modal-body" id="vote-modal-body">
    </div>
    <div class="modal-footer">
      <button class="btn btn-secondary" onclick="closeModal('vote-modal')">Cancel</button>
      <button class="btn btn-primary" id="submit-vote-btn" onclick="submitVote()"
style="flex:1;max-width:180px">
        🗳️ Submit Vote
      </button>
    </div>
  </div>
</div>

<!-- ===== RESULTS MODAL ===== -->
<div class="modal-overlay" id="results-modal">
  <div class="modal">
    <div class="modal-header">
      <div class="modal-title" id="results-modal-title">Results</div>
      <div class="modal-close" onclick="closeModal('results-modal')">✕</div>
    </div>
    <div class="modal-body" id="results-modal-body">
    </div>
    <div class="modal-footer">
      <button class="btn btn-secondary" onclick="closeModal('results-modal')">Close</button>
```

```
      </div>
    </div>
  </div>

  <!-- ===== DELETE CONFIRM MODAL ===== -->
  <div class="modal-overlay" id="confirm-modal">
    <div class="modal" style="max-width:380px">
      <div class="modal-header">
        <div class="modal-title">⚠️ Confirm Delete</div>
        <div class="modal-close" onclick="closeModal('confirm-modal')">✕</div>
      </div>
      <div class="modal-body">
        <p style="color:var(--text2);line-height:1.6">Are you sure you want to delete this poll? This
action cannot be undone and all vote data will be lost.</p>
        <div class="confirm-btns">
          <button class="btn btn-secondary"
onclick="closeModal('confirm-modal')">Cancel</button>
          <button class="btn btn-danger" id="confirm-delete-btn">🗑️ Delete</button>
        </div>
      </div>
    </div>
  </div>

</div><!-- end app -->

<script>
// ==========================================================
// DATA STORE
// ==========================================================
const COLORS = ['#6c63ff','#ff6584','#43e97b','#f7971e','#4facfe','#fa709a','#30cfd0','#a18cd1'];

let state = {
  currentUser: null,
  currentStep: 1,
  editingPollId: null,
  votingPollId: null,
  selectedOption: null,
  selectedOptions: [],
  scheduleEnabled: false,
  showResults: true,
  isPublic: true,
  searchQuery: '',
  pollsSearchQuery: ''
};
```

```
// Default users
let users = JSON.parse(localStorage.getItem('vp_users') || 'null') || [
  { id: 'u1', username: 'admin', password: 'admin123', name: 'Admin User', role: 'admin', avatar:
'#6c63ff' },
  { id: 'u2', username: 'voter1', password: 'pass123', name: 'Jane Voter', role: 'voter', avatar:
'#ff6584' },
  { id: 'u3', username: 'voter2', password: 'pass123', name: 'Bob Smith', role: 'voter', avatar:
'#43e97b' }
];

// Default polls
let polls = JSON.parse(localStorage.getItem('vp_polls') || 'null') || [
  {
    id: 'p1',
    title: 'Best Programming Language 2025',
    description: 'Which programming language do you enjoy working with most?',
    type: 'single',
    status: 'active',
    createdBy: 'u1',
    createdAt: new Date(Date.now() - 86400000 * 2).toISOString(),
    startTime: null,
    endTime: new Date(Date.now() + 86400000 * 5).toISOString(),
    showResults: true,
    isPublic: true,
    options: [
      { id: 'o1', text: 'JavaScript / TypeScript' },
      { id: 'o2', text: 'Python' },
      { id: 'o3', text: 'Rust' },
      { id: 'o4', text: 'Go' },
      { id: 'o5', text: 'Other' }
    ]
  },
  {
    id: 'p2',
    title: 'Favorite Frontend Framework',
    description: 'Which framework would you recommend in 2025?',
    type: 'single',
    status: 'active',
    createdBy: 'u1',
    createdAt: new Date(Date.now() - 86400000).toISOString(),
    startTime: null,
    endTime: new Date(Date.now() + 86400000 * 3).toISOString(),
    showResults: true,
```

```
    isPublic: true,
    options: [
      { id: 'o6', text: 'React' },
      { id: 'o7', text: 'Vue.js' },
      { id: 'o8', text: 'Angular' },
      { id: 'o9', text: 'Svelte' }
    ]
  },
  {
    id: 'p3',
    title: 'Team Meeting Day Preference',
    description: 'When should we schedule our weekly team sync?',
    type: 'single',
    status: 'closed',
    createdBy: 'u1',
    createdAt: new Date(Date.now() - 86400000 * 7).toISOString(),
    startTime: null,
    endTime: new Date(Date.now() - 86400000).toISOString(),
    showResults: true,
    isPublic: true,
    options: [
      { id: 'o10', text: 'Monday' },
      { id: 'o11', text: 'Tuesday' },
      { id: 'o12', text: 'Wednesday' },
      { id: 'o13', text: 'Thursday' },
      { id: 'o14', text: 'Friday' }
    ]
  }
];

// Votes: { pollId_userId: ['optionId', ...] }
let votes = JSON.parse(localStorage.getItem('vp_votes') || '{}');

// Seed some votes
if (!votes['p1_u2']) { votes['p1_u2'] = ['o2']; }
if (!votes['p1_u3']) { votes['p1_u3'] = ['o1']; }
if (!votes['p2_u2']) { votes['p2_u2'] = ['o6']; }
if (!votes['p3_u2']) { votes['p3_u2'] = ['o12']; }
if (!votes['p3_u3']) { votes['p3_u3'] = ['o11']; }

function save() {
  localStorage.setItem('vp_users', JSON.stringify(users));
  localStorage.setItem('vp_polls', JSON.stringify(polls));
  localStorage.setItem('vp_votes', JSON.stringify(votes));
```

```
}

// ========================================================
// AUTH
// ========================================================
function switchLoginTab(tab) {
  document.querySelectorAll('.tab-btn').forEach((b,i) => b.classList.toggle('active',
(i===0&&tab===='login')||(i===1&&tab===='register')));
  document.getElementById('login-form').style.display = tab==='login' ? '' : 'none';
  document.getElementById('register-form').style.display = tab==='register' ? '' : 'none';
}

function selectRole(el, role) {
  document.querySelectorAll('.role-option').forEach(o => o.classList.remove('selected'));
  el.classList.add('selected');
  el.dataset.selectedRole = role;
}

function doLogin() {
  const username = document.getElementById('login-username').value.trim();
  const password = document.getElementById('login-password').value;
  const user = users.find(u => u.username === username && u.password === password);
  if (!user) { showToast('Invalid username or password', 'error'); return; }
  state.currentUser = user;
  enterApp();
}

function doRegister() {
  const name = document.getElementById('reg-name').value.trim();
  const username = document.getElementById('reg-username').value.trim();
  const password = document.getElementById('reg-password').value;
  const roleEl = document.querySelector('.role-option.selected');
  const role = roleEl ? roleEl.dataset.role : 'voter';

  if (!name || !username || !password) { showToast('Please fill in all fields', 'error'); return; }
  if (password.length < 4) { showToast('Password must be at least 4 characters', 'error'); return; }
  if (users.find(u => u.username === username)) { showToast('Username already taken', 'error');
return; }

  const colors = ['#6c63ff','#ff6584','#43e97b','#f7971e','#4facfe'];
  const user = {
    id: 'u' + Date.now(),
    username, password, name, role,
    avatar: colors[Math.floor(Math.random()*colors.length)]
```

```
  };
  users.push(user);
  save();
  state.currentUser = user;
  showToast('Account created! Welcome, ' + name + '!', 'success');
  enterApp();
}

function doLogout() {
  state.currentUser = null;
  showScreen('login-screen');
  showToast('Signed out successfully', 'info');
}

function enterApp() {
  showScreen('main-screen');
  setupSidebar();
  checkPollStatuses();
  renderAll();
  showPage('dashboard');
}

// ========================================================
// SCREENS & NAVIGATION
// ========================================================
function showScreen(id) {
  document.querySelectorAll('.screen').forEach(s => s.classList.remove('active'));
  document.getElementById(id).classList.add('active');
}

function showPage(name) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.getElementById('page-' + name).classList.add('active');
  document.querySelectorAll('.nav-item').forEach(n => {
    n.classList.toggle('active', n.dataset.page === name);
  });
  closeSidebar();

  if (name === 'dashboard') renderDashboard();
  if (name === 'polls') renderPollsPage();
  if (name === 'my-votes') renderMyVotes();
  if (name === 'manage') renderManage();
  if (name === 'analytics') renderAnalytics();
  if (name === 'comparison') renderComparisonSelects();
```

```
}

function setupSidebar() {
  const u = state.currentUser;
  document.getElementById('user-display-name').textContent = u.name;
  document.getElementById('user-display-role').textContent = u.role.charAt(0).toUpperCase() +
u.role.slice(1);
  const av = document.getElementById('user-avatar');
  av.style.background = u.avatar;
  av.textContent = u.name.charAt(0).toUpperCase();

  // Show/hide admin items
  document.querySelectorAll('.admin-only').forEach(el => {
    el.style.display = u.role === 'admin' ? '' : 'none';
  });
}

function openSidebar() {
  document.getElementById('sidebar').classList.add('open');
  document.getElementById('sidebar-overlay').classList.add('open');
}

function closeSidebar() {
  document.getElementById('sidebar').classList.remove('open');
  document.getElementById('sidebar-overlay').classList.remove('open');
}

// ========================================================
// POLL STATUS CHECK (auto-close)
// ========================================================
function checkPollStatuses() {
  const now = new Date();
  let changed = false;
  polls.forEach(poll => {
    if (poll.status === 'active' && poll.endTime && new Date(poll.endTime) < now) {
      poll.status = 'closed';
      changed = true;
    }
    if (poll.status === 'draft' || poll.status === 'scheduled') {
      if (poll.startTime && new Date(poll.startTime) <= now && poll.status !== 'closed') {
        poll.status = 'active';
        changed = true;
      }
    }
```

```
  });
  if (changed) save();
}

// Run every 30 seconds
setInterval(() => { checkPollStatuses(); renderAll(); }, 30000);

// ========================================================
// RENDER FUNCTIONS
// ========================================================
function renderAll() {
  updateStats();
  updateNavBadge();
}

function updateNavBadge() {
  const active = polls.filter(p => getPollStatus(p) === 'active').length;
  document.getElementById('active-poll-count').textContent = active;
}

function updateStats() {
  const u = state.currentUser;
  if (!u) return;
  const activePollsCount = polls.filter(p => getPollStatus(p) === 'active').length;
  const myVoteCount = Object.keys(votes).filter(k => k.endsWith('_' + u.id)).length;
  const totalVoteCount = Object.keys(votes).length;
  document.getElementById('stat-total').textContent = polls.length;
  document.getElementById('stat-active').textContent = activePollsCount;
  document.getElementById('stat-voted').textContent = myVoteCount;
  document.getElementById('stat-votes-total').textContent = totalVoteCount;
  document.getElementById('dash-subtitle').textContent = 'Welcome back, ' + u.name + '!';
}

function getPollStatus(poll) {
  const now = new Date();
  if (poll.status === 'draft') return 'draft';
  if (poll.status === 'closed') return 'closed';
  if (poll.endTime && new Date(poll.endTime) < now) return 'closed';
  if (poll.startTime && new Date(poll.startTime) > now) return 'scheduled';
  return 'active';
}

function getVoteCount(poll) {
  return Object.keys(votes).filter(k => k.startsWith(poll.id + '_')).length;
```

```
}

function getTotalVotes(poll) {
  // Sum all option votes
  let total = 0;
  poll.options.forEach(opt => {
    Object.values(votes).forEach(v => {
      if (v.includes(opt.id)) total++;
    });
  });
  return total;
}

function getOptionVoteCount(poll, optionId) {
  let count = 0;
  Object.entries(votes).forEach(([key, optIds]) => {
    if (key.startsWith(poll.id + '_') && optIds.includes(optionId)) count++;
  });
  return count;
}

function hasVoted(poll) {
  return !!votes[poll.id + '_' + state.currentUser.id];
}

function getTimeRemaining(endTime) {
  const now = new Date();
  const end = new Date(endTime);
  const diff = end - now;
  if (diff <= 0) return null;
  const days = Math.floor(diff / 86400000);
  const hours = Math.floor((diff % 86400000) / 3600000);
  const mins = Math.floor((diff % 3600000) / 60000);
  if (days > 0) return days + 'd ' + hours + 'h remaining';
  if (hours > 0) return hours + 'h ' + mins + 'm remaining';
  return mins + 'm remaining';
}

function formatDate(iso) {
  if (!iso) return '';
  return new Date(iso).toLocaleDateString('en-US', { month:'short', day:'numeric', year:'numeric',
hour:'2-digit', minute:'2-digit' });
}
```

```
function pollCardHTML(poll, context) {
  const status = getPollStatus(poll);
  const voteCount = getVoteCount(poll);
  const voted = hasVoted(poll);
  const u = state.currentUser;
  const remaining = poll.endTime ? getTimeRemaining(poll.endTime) : null;
  const maxVotes = Math.max(...poll.options.map(o => getOptionVoteCount(poll, o.id)), 1);
  const progress = voteCount > 0 ? Math.min(100, (voteCount / Math.max(voteCount, 10)) *
100) : 0;

  const statusLabel = {active:'Active', closed:'Closed', scheduled:'Scheduled',
draft:'Draft'}[status];

  let actionBtns = '';

  if (context === 'manage' && u.role === 'admin') {
    actionBtns = `
      <div class="poll-actions">
        <button class="btn btn-icon btn-secondary btn-sm" onclick="openEditPoll('${poll.id}')"
title="Edit">✏️</button>
        <button class="btn btn-icon btn-danger btn-sm" onclick="confirmDelete('${poll.id}')"
title="Delete">🗑️</button>
      </div>`;
  }

  let footerBtns = '';
  if (context === 'manage') {
    footerBtns = `
      <button class="btn btn-secondary btn-sm" onclick="openResults('${poll.id}')">📊
Results</button>
      <span class="vote-count"><strong>${voteCount}</strong> voters</span>`;
  } else {
    if (status === 'active' && !voted) {
      footerBtns = `<button class="btn btn-primary btn-sm" onclick="openVote('${poll.id}')">🗳️
Vote Now</button>`;
    } else if (voted) {
      footerBtns = `
        <span class="time-pill success">✅ You voted</span>
        ${poll.showResults || u.role==='admin' ? `<button class="btn btn-secondary btn-sm"
onclick="openResults('${poll.id}')">📊 Results</button>` : ''}`;
    } else if (status === 'closed') {
      footerBtns = `<button class="btn btn-secondary btn-sm"
onclick="openResults('${poll.id}')">📊 View Results</button>`;
    } else if (status === 'scheduled') {
```

```
      footerBtns = `<span class="time-pill">🕐 Starts ${formatDate(poll.startTime)}</span>`;
    }
  }

  return `
    <div class="poll-card" id="poll-card-${poll.id}">
      <div class="poll-card-header">
        <div>
          <span class="status-badge ${status}">${statusLabel}</span>
        </div>
        ${actionBtns}
      </div>
      <div class="poll-title">${escHtml(poll.title)}</div>
      ${poll.description ? `<div class="poll-desc">${escHtml(poll.description)}</div>` : ''}
      <div class="poll-meta">
        <span class="poll-meta-item">👤 ${escHtml(getUserName(poll.createdBy))}</span>
        <span class="poll-meta-item">📋 ${poll.options.length} options</span>
        <span class="poll-meta-item">🗳️ ${voteCount} voters</span>
        ${poll.type === 'multiple' ? '<span class="poll-meta-item">☑ Multi-choice</span>' : ''}
      </div>
      <div class="poll-progress"><div class="poll-progress-bar"
style="width:${progress}%"></div></div>
      <div class="poll-footer">
        <div style="display:flex;gap:8px;align-items:center;flex-wrap:wrap">
          ${footerBtns}
        </div>
        ${remaining ? `<span class="time-pill ${remaining.includes('m remaining') &&
!remaining.includes('h') ? 'danger' : ''}">${remaining}</span>` : (status==='closed' ? '<span
class="time-pill">Poll closed</span>' : '')}
      </div>
    </div>`;
}

function renderDashboard() {
  checkPollStatuses();
  updateStats();
  const activePollsEl = document.getElementById('dashboard-polls');
  const q = state.searchQuery.toLowerCase();
  const active = polls.filter(p => {
    const st = getPollStatus(p);
    if (st !== 'active') return false;
    if (q && !p.title.toLowerCase().includes(q) && !p.description.toLowerCase().includes(q)) return
false;
    return p.isPublic || state.currentUser.role === 'admin';
```

```
  });

  if (active.length === 0) {
    activePollsEl.innerHTML = `<div class="empty-state" style="grid-column:1/-1">
      <div class="empty-icon">🗳️</div>
      <div class="empty-title">No active polls</div>
      <div class="empty-desc">There are no active polls right now. Check back later!</div>
    </div>`;
    return;
  }
  activePollsEl.innerHTML = active.map(p => pollCardHTML(p,'dashboard')).join('');
}

function renderPollsPage() {
  checkPollStatuses();
  const q = (document.querySelector('#page-polls .search-input')?.value || '').toLowerCase();
  const statusF = document.getElementById('status-filter')?.value || 'all';
  const sortF = document.getElementById('sort-filter')?.value || 'newest';
  const u = state.currentUser;

  let filtered = polls.filter(p => {
    if (!p.isPublic && u.role !== 'admin') return false;
    const st = getPollStatus(p);
    if (statusF !== 'all' && st !== statusF) return false;
    if (q && !p.title.toLowerCase().includes(q) && !(p.description||'').toLowerCase().includes(q))
return false;
    return true;
  });

  filtered.sort((a, b) => {
    if (sortF === 'newest') return new Date(b.createdAt) - new Date(a.createdAt);
    if (sortF === 'oldest') return new Date(a.createdAt) - new Date(b.createdAt);
    if (sortF === 'most-votes') return getVoteCount(b) - getVoteCount(a);
    return 0;
  });

  const el = document.getElementById('polls-list');
  if (filtered.length === 0) {
    el.innerHTML = `<div class="empty-state" style="grid-column:1/-1">
      <div class="empty-icon">🔍</div>
      <div class="empty-title">No polls found</div>
      <div class="empty-desc">Try adjusting your search or filters.</div>
    </div>`;
    return;
```

```
  }
  el.innerHTML = filtered.map(p => pollCardHTML(p,'polls')).join('');
}

function renderMyVotes() {
  const u = state.currentUser;
  const votedPollIds = Object.keys(votes)
    .filter(k => k.endsWith('_' + u.id))
    .map(k => k.split('_')[0]);
  const votedPolls = polls.filter(p => votedPollIds.includes(p.id));
  const el = document.getElementById('my-votes-list');
  if (votedPolls.length === 0) {
    el.innerHTML = `<div class="empty-state" style="grid-column:1/-1">
      <div class="empty-icon">✅</div>
      <div class="empty-title">No votes yet</div>
      <div class="empty-desc">You haven't voted in any polls yet. Head to "All Polls" to get
started!</div>
    </div>`;
    return;
  }
  el.innerHTML = votedPolls.map(p => pollCardHTML(p,'my-votes')).join('');
}

function renderManage() {
  checkPollStatuses();
  const u = state.currentUser;
  const el = document.getElementById('manage-list');
  const adminPolls = polls.filter(p => p.createdBy === u.id || u.role === 'admin');
  if (adminPolls.length === 0) {
    el.innerHTML = `<div class="empty-state" style="grid-column:1/-1">
      <div class="empty-icon">📋</div>
      <div class="empty-title">No polls yet</div>
      <div class="empty-desc">Create your first poll to get started!</div>
    </div>`;
    return;
  }
  el.innerHTML = adminPolls.map(p => pollCardHTML(p,'manage')).join('');
}

function renderAnalytics() {
  const el = document.getElementById('analytics-content');
  if (polls.length === 0) {
    el.innerHTML = `<div class="empty-state"><div class="empty-icon">📈</div><div
class="empty-title">No data yet</div></div>`;
```

```
    return;
  }

  el.innerHTML = `<div class="analytics-grid">${polls.map(poll => {
    const total = getVoteCount(poll);
    const status = getPollStatus(poll);
    return `<div class="analytics-card">
      <div
style="display:flex;justify-content:space-between;align-items:flex-start;gap:8px;margin-bottom:6
px">
        <div class="analytics-card-title">${escHtml(poll.title)}</div>
        <span class="status-badge ${status}">${status}</span>
      </div>
      <div style="font-size:13px;color:var(--text2);margin-bottom:16px">${total} total voters</div>
      <div class="results-container">
        ${poll.options.map((opt, i) => {
          const cnt = getOptionVoteCount(poll, opt.id);
          const pct = total > 0 ? Math.round(cnt / total * 100) : 0;
          return `<div class="result-row">
            <div class="result-header">
              <span class="result-label">${escHtml(opt.text)}</span>
              <div style="display:flex;gap:8px;align-items:center">
                <span class="result-count">${cnt}</span>
                <span class="result-pct">${pct}%</span>
              </div>
            </div>
            <div class="result-bar-bg">
              <div class="result-bar bar-color-${i%8}" style="width:${pct}%"></div>
            </div>
          </div>`;
        }).join('')}
      </div>
      ${renderPieChart(poll)}
    </div>`;
  }).join('')}</div>`;
}

function renderPieChart(poll) {
  const total = getTotalVotes(poll);
  if (total === 0) return '<p style="color:var(--text3);font-size:13px;margin-top:12px">No votes
cast yet</p>';

  let cumulativePct = 0;
  const segments = poll.options.map((opt, i) => {
```

```
    const cnt = getOptionVoteCount(poll, opt.id);
    const pct = total > 0 ? (cnt / total) * 100 : 0;
    const start = cumulativePct;
    cumulativePct += pct;
    return { opt, cnt, pct, start };
  });

  const size = 120;
  const cx = size/2, cy = size/2, r = size/2 - 4;

  function polarToCartesian(angle) {
    const rad = (angle - 90) * Math.PI / 180;
    return { x: cx + r * Math.cos(rad), y: cy + r * Math.sin(rad) };
  }

  const svgSegments = segments.map((seg, i) => {
    if (seg.pct === 0) return '';
    if (seg.pct >= 99.9) {
      return `<circle cx="${cx}" cy="${cy}" r="${r}" fill="${COLORS[i%8]}"/>`;
    }
    const startAngle = seg.start * 3.6;
    const endAngle = (seg.start + seg.pct) * 3.6;
    const s = polarToCartesian(startAngle);
    const e = polarToCartesian(endAngle);
    const lg = seg.pct > 50 ? 1 : 0;
    return `<path d="M${cx},${cy} L${s.x},${s.y} A${r},${r} 0 ${lg},1 ${e.x},${e.y} Z"
fill="${COLORS[i%8]}" opacity="0.9"/>`;
  }).join('');

  const legendItems = segments.filter(s => s.pct > 0).map((seg, i) => {
    const idx = poll.options.indexOf(seg.opt);
    return `<div class="legend-item"><div class="legend-dot"
style="background:${COLORS[idx%8]}"></div><span
style="flex:1;min-width:0;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">${escHtml(
seg.opt.text)}</span><strong>${Math.round(seg.pct)}%</strong></div>`;
  }).join('');

  return `<div class="chart-container" style="margin-top:16px">
    <div class="chart-wrap" style="flex:0 0 auto">
      <svg width="${size}" height="${size}" viewBox="0 0 ${size} ${size}"
style="border-radius:50%;overflow:visible">
        ${svgSegments}
        <circle cx="${cx}" cy="${cy}" r="${r*0.45}" fill="var(--surface)"/>
```

```
      <text x="${cx}" y="${cy+5}" text-anchor="middle" fill="var(--text2)" font-size="12"
font-family="Syne,sans-serif">${total}</text>
    </svg>
  </div>
  <div class="chart-legend">${legendItems}</div>
 </div>`;
}

function renderComparisonSelects() {
 const opts = polls.map(p => `<option value="${p.id}">${escHtml(p.title)}</option>`).join('');
 document.getElementById('compare-poll-1').innerHTML = '<option value="">Select Poll
1…</option>' + opts;
 document.getElementById('compare-poll-2').innerHTML = '<option value="">Select Poll
2…</option>' + opts;
 document.getElementById('comparison-content').innerHTML = '';
}

function renderComparison() {
 const id1 = document.getElementById('compare-poll-1').value;
 const id2 = document.getElementById('compare-poll-2').value;
 if (!id1 || !id2) { showToast('Please select two polls to compare', 'error'); return; }
 if (id1 === id2) { showToast('Please select two different polls', 'error'); return; }

 const p1 = polls.find(p => p.id === id1);
 const p2 = polls.find(p => p.id === id2);

 const pollBlock = (poll) => {
  const total = getVoteCount(poll);
  const status = getPollStatus(poll);
  return `<div class="analytics-card">
   <div class="analytics-card-title">${escHtml(poll.title)}</div>
   <div style="display:flex;gap:8px;margin-bottom:16px;flex-wrap:wrap">
    <span class="status-badge ${status}">${status}</span>
    <span class="time-pill">🗳️ ${total} voters</span>
    <span class="time-pill">📋 ${poll.options.length} options</span>
   </div>
   <div class="results-container">
    ${poll.options.map((opt, i) => {
     const cnt = getOptionVoteCount(poll, opt.id);
     const pct = total > 0 ? Math.round(cnt / total * 100) : 0;
     return `<div class="result-row">
      <div class="result-header">
       <span class="result-label">${escHtml(opt.text)}</span>
       <span class="result-pct">${pct}%</span>
```

```
          </div>
          <div class="result-bar-bg">
            <div class="result-bar bar-color-${i%8}" style="width:${pct}%"></div>
          </div>
        </div>`;
      }).join('')}
    </div>
    ${renderPieChart(poll)}
  </div>`;
};

document.getElementById('comparison-content').innerHTML = `
  <div class="comparison-grid">
    ${pollBlock(p1)}
    ${pollBlock(p2)}
  </div>`;
}

// ========================================================
// VOTING
// ========================================================
function openVote(pollId) {
  const poll = polls.find(p => p.id === pollId);
  if (!poll) return;

  const status = getPollStatus(poll);
  if (status !== 'active') { showToast('This poll is not active', 'error'); return; }
  if (hasVoted(poll)) { showToast('You have already voted in this poll!', 'error'); return; }

  state.votingPollId = pollId;
  state.selectedOption = null;
  state.selectedOptions = [];

  document.getElementById('vote-modal-title').textContent = '🗳️ ' + poll.title;

  const body = document.getElementById('vote-modal-body');
  body.innerHTML = `
    <div class="info-box">
      <strong>Instructions:</strong> ${poll.type === 'multiple' ? 'Select one or more options.' :
'Select exactly one option.'} Your vote is final and cannot be changed.
      ${poll.endTime ? `<br>⏰ Poll closes: <strong>${formatDate(poll.endTime)}</strong>` : ''}
    </div>
```

```
    ${poll.description ? `<p
style="color:var(--text2);font-size:14px;margin-bottom:16px">${escHtml(poll.description)}</p>` :
''}
    <div class="vote-options">
      ${poll.options.map(opt => `
        <div class="vote-option" data-id="${opt.id}"
onclick="selectVoteOption(this,'${opt.id}',${poll.type==='multiple'})">
          <div class="vote-radio"></div>
          <div class="vote-option-text">${escHtml(opt.text)}</div>
        </div>`).join('')}
    </div>`;

  openModal('vote-modal');
}

function selectVoteOption(el, optId, isMultiple) {
  if (isMultiple) {
    el.classList.toggle('selected');
    const idx = state.selectedOptions.indexOf(optId);
    if (idx > -1) state.selectedOptions.splice(idx, 1);
    else state.selectedOptions.push(optId);
  } else {
    document.querySelectorAll('.vote-option').forEach(o => o.classList.remove('selected'));
    el.classList.add('selected');
    state.selectedOption = optId;
  }
}

function submitVote() {
  const poll = polls.find(p => p.id === state.votingPollId);
  if (!poll) return;

  const status = getPollStatus(poll);
  if (status !== 'active') { showToast('This poll has closed', 'error'); closeModal('vote-modal');
return; }
  if (hasVoted(poll)) { showToast('You have already voted!', 'error'); closeModal('vote-modal');
return; }

  let selected;
  if (poll.type === 'multiple') {
    selected = state.selectedOptions;
    if (selected.length === 0) { showToast('Please select at least one option', 'error'); return; }
  } else {
    if (!state.selectedOption) { showToast('Please select an option', 'error'); return; }
```

```
    selected = [state.selectedOption];
  }

  const key = poll.id + '_' + state.currentUser.id;
  votes[key] = selected;
  save();

  closeModal('vote-modal');
  showToast('✅ Your vote has been recorded!', 'success');
  renderAll();

  // Auto-show results if allowed
  setTimeout(() => {
    if (poll.showResults) openResults(poll.id);
  }, 400);

  // Refresh current page
  const activePage = document.querySelector('.page.active')?.id?.replace('page-', '');
  if (activePage) {
    if (activePage === 'dashboard') renderDashboard();
    if (activePage === 'polls') renderPollsPage();
    if (activePage === 'manage') renderManage();
  }
}

function openResults(pollId) {
  const poll = polls.find(p => p.id === pollId);
  if (!poll) return;
  const total = getVoteCount(poll);
  const u = state.currentUser;
  const voted = hasVoted(poll);
  const myVote = voted ? votes[poll.id + '_' + u.id] : null;

  if (!poll.showResults && u.role !== 'admin') {
    showToast('Results are not public for this poll', 'info');
    return;
  }

  document.getElementById('results-modal-title').textContent = '📊 ' + poll.title;
  const body = document.getElementById('results-modal-body');

  const leadingOpt = poll.options.reduce((best, opt) => {
    const cnt = getOptionVoteCount(poll, opt.id);
    return cnt > (getOptionVoteCount(poll, best.id)||0) ? opt : best;
```

```
  }, poll.options[0]);

  body.innerHTML = `
    <div style="display:flex;gap:12px;margin-bottom:20px;flex-wrap:wrap">
      <span class="time-pill success">🗳️ ${total} total voters</span>
      ${voted ? '<span class="time-pill success">✅ You voted</span>' : ''}
    </div>
    ${total > 0 ? `<div style="font-size:13px;color:var(--text2);margin-bottom:16px">🏆 Leading:
<strong style="color:var(--text)">${escHtml(leadingOpt.text)}</strong></div>` : ''}
    <div class="results-container">
      ${poll.options.map((opt, i) => {
        const cnt = getOptionVoteCount(poll, opt.id);
        const pct = total > 0 ? Math.round(cnt / total * 100) : 0;
        const isMyVote = myVote && myVote.includes(opt.id);
        const isLeading = opt.id === leadingOpt.id && total > 0;
        return `<div class="result-row">
          <div class="result-header">
            <span class="result-label">
              ${isLeading && cnt > 0 ? '🏆 ' : ''}
              ${escHtml(opt.text)}
              ${isMyVote ? '<span style="color:var(--success);font-size:12px;margin-left:6px">←
Your vote</span>' : ''}
            </span>
            <div style="display:flex;gap:8px;align-items:center">
              <span class="result-count">${cnt} votes</span>
              <span class="result-pct">${pct}%</span>
            </div>
          </div>
          <div class="result-bar-bg">
            <div class="result-bar bar-color-${i%8}" style="width:${pct}%"></div>
          </div>
        </div>`;
      }).join('')}
    </div>
    ${renderPieChart(poll)}`;

  openModal('results-modal');
}

// ==========================================================
// POLL CRUD
// ==========================================================
let modalStep = 1;
```

```
function openCreatePoll() {
  state.editingPollId = null;
  modalStep = 1;
  state.scheduleEnabled = false;
  state.showResults = true;
  state.isPublic = true;

  document.getElementById('poll-modal-title').textContent = 'Create New Poll';
  document.getElementById('poll-title').value = '';
  document.getElementById('poll-desc').value = '';
  document.getElementById('poll-type').value = 'single';
  document.getElementById('poll-status').value = 'active';
  document.getElementById('poll-start').value = '';
  document.getElementById('poll-end').value = '';
  document.getElementById('options-list').innerHTML = '';
  addOption(); addOption(); // start with 2 options

  document.getElementById('schedule-switch').className = 'switch';
  document.getElementById('results-switch').className = 'switch on';
  document.getElementById('public-switch').className = 'switch on';
  document.getElementById('schedule-fields').style.display = 'none';

  goToStep(1);
  openModal('poll-modal');
}

function openEditPoll(pollId) {
  const poll = polls.find(p => p.id === pollId);
  if (!poll) return;
  state.editingPollId = pollId;
  state.scheduleEnabled = !!(poll.startTime || poll.endTime);
  state.showResults = poll.showResults !== false;
  state.isPublic = poll.isPublic !== false;

  document.getElementById('poll-modal-title').textContent = 'Edit Poll';
  document.getElementById('poll-title').value = poll.title;
  document.getElementById('poll-desc').value = poll.description || '';
  document.getElementById('poll-type').value = poll.type || 'single';
  document.getElementById('poll-status').value = poll.status;

  // Schedule
  document.getElementById('schedule-switch').className = state.scheduleEnabled ? 'switch
on' : 'switch';
  document.getElementById('schedule-fields').style.display = state.scheduleEnabled ? '' : 'none';
```

```
  if (poll.startTime) document.getElementById('poll-start').value = poll.startTime.slice(0,16);
  if (poll.endTime) document.getElementById('poll-end').value = poll.endTime.slice(0,16);

  document.getElementById('results-switch').className = state.showResults ? 'switch on' :
'switch';
  document.getElementById('public-switch').className = state.isPublic ? 'switch on' : 'switch';

  // Options
  const ol = document.getElementById('options-list');
  ol.innerHTML = '';
  poll.options.forEach(opt => addOption(opt.text));

  goToStep(1);
  openModal('poll-modal');
}

function goToStep(step) {
  modalStep = step;
  document.getElementById('modal-step-1').style.display = step===1 ? '' : 'none';
  document.getElementById('modal-step-2').style.display = step===2 ? '' : 'none';
  document.getElementById('modal-step-3').style.display = step===3 ? '' : 'none';

  document.getElementById('modal-back-btn').style.display = step > 1 ? '' : 'none';
  document.getElementById('modal-next-btn').textContent = step === 3 ? '💾 Save Poll' : 'Next
→';

  for (let i = 1; i <= 3; i++) {
    const el = document.getElementById('step-' + i);
    el.classList.remove('active','done');
    if (i < step) el.classList.add('done');
    if (i === step) el.classList.add('active');
  }
}

function pollModalNext() {
  if (modalStep === 1) {
    const title = document.getElementById('poll-title').value.trim();
    if (!title) { showToast('Poll title is required', 'error'); return; }
    goToStep(2);
  } else if (modalStep === 2) {
    const inputs = document.querySelectorAll('#options-list .option-text');
    const texts = [...inputs].map(i => i.value.trim()).filter(Boolean);
    if (texts.length < 2) { showToast('Please add at least 2 options', 'error'); return; }
    goToStep(3);
```

```
  } else if (modalStep === 3) {
    savePoll();
  }
}

function pollModalBack() {
  goToStep(modalStep - 1);
}

function toggleSchedule() {
  state.scheduleEnabled = !state.scheduleEnabled;
  document.getElementById('schedule-switch').className = state.scheduleEnabled ? 'switch
on' : 'switch';
  document.getElementById('schedule-fields').style.display = state.scheduleEnabled ? '' : 'none';
}

function toggleResults() {
  state.showResults = !state.showResults;
  document.getElementById('results-switch').className = state.showResults ? 'switch on' :
'switch';
}

function togglePublic() {
  state.isPublic = !state.isPublic;
  document.getElementById('public-switch').className = state.isPublic ? 'switch on' : 'switch';
}

function addOption(defaultText = '') {
  const ol = document.getElementById('options-list');
  const id = 'opt-' + Date.now() + '-' + Math.random().toString(36).slice(2);
  const div = document.createElement('div');
  div.className = 'option-item';
  div.innerHTML = `
    <input class="form-input option-text" placeholder="Option text…"
value="${escHtml(defaultText)}">
    <button class="btn btn-danger btn-icon btn-sm" onclick="this.closest('.option-item').remove()"
title="Remove">✕</button>`;
  ol.appendChild(div);
  div.querySelector('input').focus();
}

function savePoll() {
  const title = document.getElementById('poll-title').value.trim();
  const desc = document.getElementById('poll-desc').value.trim();
```

```
const type = document.getElementById('poll-type').value;
const status = document.getElementById('poll-status').value;
const startVal = document.getElementById('poll-start').value;
const endVal = document.getElementById('poll-end').value;

const inputs = document.querySelectorAll('#options-list .option-text');
const optionTexts = [...inputs].map(i => i.value.trim()).filter(Boolean);

if (optionTexts.length < 2) { showToast('At least 2 options required', 'error'); return; }

if (state.scheduleEnabled && startVal && endVal) {
  if (new Date(endVal) <= new Date(startVal)) {
    showToast('End time must be after start time', 'error');
    return;
  }
}

if (state.editingPollId) {
  const poll = polls.find(p => p.id === state.editingPollId);
  const existingOptions = poll.options;

  poll.title = title;
  poll.description = desc;
  poll.type = type;
  poll.status = status;
  poll.startTime = state.scheduleEnabled && startVal ? new Date(startVal).toISOString() : null;
  poll.endTime = state.scheduleEnabled && endVal ? new Date(endVal).toISOString() : null;
  poll.showResults = state.showResults;
  poll.isPublic = state.isPublic;

  // Preserve existing options where possible
  poll.options = optionTexts.map((text, i) => {
    return existingOptions[i] ? { ...existingOptions[i], text } : { id: 'o' + Date.now() + i, text };
  });

  showToast('Poll updated successfully!', 'success');
} else {
  const poll = {
    id: 'p' + Date.now(),
    title, description: desc, type, status,
    createdBy: state.currentUser.id,
    createdAt: new Date().toISOString(),
    startTime: state.scheduleEnabled && startVal ? new Date(startVal).toISOString() : null,
    endTime: state.scheduleEnabled && endVal ? new Date(endVal).toISOString() : null,
```

```
      showResults: state.showResults,
      isPublic: state.isPublic,
      options: optionTexts.map((text, i) => ({ id: 'o' + Date.now() + i, text }))
    };
    polls.unshift(poll);
    showToast('Poll created successfully!', 'success');
  }

  save();
  closeModal('poll-modal');
  renderAll();
  renderManage();
  renderAnalytics();
}

function confirmDelete(pollId) {
  document.getElementById('confirm-delete-btn').onclick = () => {
    polls = polls.filter(p => p.id !== pollId);
    // Remove all votes for this poll
    Object.keys(votes).forEach(k => { if (k.startsWith(pollId + '_')) delete votes[k]; });
    save();
    closeModal('confirm-modal');
    showToast('Poll deleted', 'info');
    renderAll();
    renderManage();
    renderAnalytics();
  };
  openModal('confirm-modal');
}

// ========================================================
// SEARCH
// ========================================================
function filterPolls(query, context) {
  if (context === 'dashboard') { state.searchQuery = query; renderDashboard(); }
  if (context === 'polls') renderPollsPage();
}

// ========================================================
// MODAL HELPERS
// ========================================================
function openModal(id) {
  document.getElementById(id).classList.add('open');
  document.body.style.overflow = 'hidden';
```

```
}

function closeModal(id) {
  document.getElementById(id).classList.remove('open');
  document.body.style.overflow = '';
}

// Close modals on overlay click
document.querySelectorAll('.modal-overlay').forEach(overlay => {
  overlay.addEventListener('click', e => {
    if (e.target === overlay) closeModal(overlay.id);
  });
});

// ========================================================
// TOAST
// ========================================================
function showToast(message, type = 'info') {
  const icons = { success: '✅', error: '❌', info: 'ℹ️', warning: '⚠️' };
  const container = document.getElementById('toast-container');
  const toast = document.createElement('div');
  toast.className = `toast ${type}`;
  toast.innerHTML = `<span>${icons[type]}</span>${escHtml(message)}`;
  container.appendChild(toast);
  setTimeout(() => toast.remove(), 3100);
}

// ========================================================
// HELPERS
// ========================================================
function escHtml(str) {
  if (!str) return '';
  return String(str)
    .replace(/&/g,'&amp;')
    .replace(/</g,'&lt;')
    .replace(/>/g,'&gt;')
    .replace(/"/g,'&quot;')
    .replace(/'/g,'&#39;');
}

function getUserName(userId) {
  const u = users.find(u => u.id === userId);
  return u ? u.name : 'Unknown';
}
```

```
// ========================================================
// KEYBOARD SHORTCUTS
// ========================================================
document.addEventListener('keydown', e => {
  if (e.key === 'Escape') {
    document.querySelectorAll('.modal-overlay.open').forEach(m => closeModal(m.id));
  }
  if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
    const openModal = document.querySelector('.modal-overlay.open');
    if (openModal?.id === 'poll-modal') pollModalNext();
    if (openModal?.id === 'vote-modal') submitVote();
  }
});

// ========================================================
// ENTER to login
// ========================================================
document.addEventListener('keydown', e => {
  if (e.key === 'Enter' && document.getElementById('login-screen').classList.contains('active')) {
    const loginForm = document.getElementById('login-form');
    const regForm = document.getElementById('register-form');
    if (loginForm.style.display !== 'none') doLogin();
    else if (regForm.style.display !== 'none') doRegister();
  }
});

// ========================================================
// INIT
// ========================================================
save(); // persist defaults
</script>
</body>
</html>
```